Guia de interface — CRUD na tela com fetch

Disciplina: Programação III

Curso: Ensino Médio Integrado — Técnico em Informática

Professor: Elder Francisco Fontana Bernardi

Instituição: IFSul — Campus Passo Fundo

Versão 3 · 10/11/2026. Revisão de linguagem. O código do exemplo e o conteúdo técnico são os mesmos da versão 2.

Página do trabalho · Enunciado

Este guia traz um exemplo completo, pronto para rodar. O exemplo mostra como uma página HTML troca dados com uma API Express usando `fetch`. A página lista, cadastra, edita e exclui registros sem recarregar. No fim, o guia indica como aplicar cada padrão às músicas do projeto. O guia não entrega o código das músicas.

Sumário

1. Para que serve
2. Rodar o exemplo
3. Padrão 1 — HTML e JavaScript se encontram pelos IDs
4. Padrão 2 — Listar e desenhar
5. Padrão 3 — Cadastrar
6. Padrão 4 — Erro HTTP × falha de conexão
7. Padrão 5 — Selecionar e editar
8. Padrão 6 — Excluir com confirmação
9. Padrão 7 — Evitar envio duplicado
10. Conferir na aba Rede (DevTools)
11. Aplicando às músicas (sem código)
12. Se deu errado

Para que serve

No projeto integrador, "O repertório da turma", quatro requisitos acontecem **na tela**: R2 (cadastrar), R3 (editar), R4 (excluir) e R5 (validar e informar). Na tela, a pessoa digita, clica e vê o resultado. A API de músicas já responde. Falta a parte da página que chama a API, interpreta a resposta e atualiza o que aparece.

O guia usa um exemplo menor, o **Mural de recados**, em que cada recado tem só um texto. Com um campo só, o código de cada padrão fica curto e mais fácil de ler. As músicas têm mais campos. Depois de ler o mural, vocês aplicam os mesmos padrões ao catálogo de músicas, com duas diferenças que a seção "Aplicando às músicas" explica.

O que o guia entrega e o que não entrega:

Entrega	Não entrega
O mural completo, com cada arquivo comentado	O código de cadastro, edição e exclusão de músicas
Os sete padrões que se repetem em qualquer tela com API	A decisão de como organizar o código de vocês
Como conferir cada operação na aba Rede do navegador	A resposta pronta do Percurso para testar e demonstrar
Tabelas e checklists para as músicas	

> **Para quem é cada trilha.** Há dois pontos de partida possíveis:
>
> - a base de retomada, que é o projeto que acompanha o Roteiro de retomada (disponível no Classroom da disciplina);
> - o projeto de quem seguiu o "Guia passo a passo do Supabase" (disponível no Classroom da disciplina).
>
> Nos dois casos, a tela final é a mesma. A seção "Aplicando às músicas" tem uma parte para cada caso.

↑ Voltar ao sumário

Rodar o exemplo

O mural está na pasta `mural_recados`. Essa pasta fica dentro do pacote de materiais de apoio, disponível no Classroom da disciplina. Vocês também podem baixar só o mural em mural_recados.zip, na página do trabalho.

Para rodar:

1. Descompactem o pacote.
2. Abram a pasta `mural_recados` no VS Code.
3. Abram o terminal integrado.
4. Digitem no terminal os dois comandos abaixo, nesta ordem:

```
npm install
npm start
```

O terminal deve mostrar `Mural no ar: http://localhost:3001`. Abram esse endereço no navegador. A página lista um recado inicial ("Ensaio do karaokê na sexta.") e já aceita cadastrar, editar e excluir.

Os recados ficam num array na memória do servidor. Se vocês pararem o servidor e iniciarem de novo, a lista volta a ter só o recado inicial. Isso é esperado. O mural serve para ensinar a tela. Ele não ensina a persistência. As músicas do projeto ficam gravadas no banco.

Árvore de arquivos

```
mural_recados/
├── package.json
├── app.js
└── public/
    ├── index.html
    ├── estilo.css
    └── mural.js
```

Quem usa quem

Arquivo	Usa	Para quê
`public/index.html`	`estilo.css` e `mural.js`	A estrutura da página: formulário, lista, mensagem.
`public/mural.js`	os elementos de `index.html`, pelos IDs; a rota `/api/recados`	Lê o que a pessoa digitou, chama a API e desenha o resultado.
`app.js`	`express`; a pasta `public`	Serve os arquivos da pasta `public` e responde às rotas `/api/recados`.
`package.json`	—	Declara a dependência (`express`) e o comando `npm start`.

A sequência de uma ação, do clique ao resultado:

1. `index.html` mostra o botão.
2. `mural.js` trata o clique e chama `fetch('/api/recados')`.
3. `app.js` recebe a requisição, altera o array e responde.
4. `mural.js` redesenha a lista com a resposta.

Os cinco arquivos, por inteiro

Os arquivos abaixo são iguais aos do pacote. Leiam cada arquivo uma vez antes de seguir para os padrões. As seções seguintes explicam trechos específicos desses arquivos.

Arquivo `package.json`

```
{
  "name": "mural-recados",
  "version": "1.0.0",
  "private": true,
  "description": "Exemplo de CRUD na tela com fetch — Programação III, IFSul Passo Fundo",
  "type": "commonjs",
  "main": "app.js",
  "scripts": { "start": "node app.js" },
  "dependencies": { "express": "5.2.1" },
  "engines": { "node": "⩾22" }
}
```

O `package.json` declara só o `express`, na versão exata, e exige Node 22 ou superior. O comando `npm start` roda `node app.js`.

Arquivo `app.js`

```
// mural_recados/app.js — API de exemplo: recados guardados num array
(memória).
const express = require('express');
const path = require('node:path');

const app = express();
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

let recados = [{ id: 1, texto: 'Ensaio do karaokê na sexta.' }];
let proximoId = 2;

// Lê o texto do corpo; devolve null se faltar ou vier vazio
function lerTexto(body) {
    const texto = typeof body?.texto === 'string' ? body.texto.trim() : '';
    return texto || null;
}

app.get('/api/recados', (req, res) => {
    res.json(recados);
});

app.get('/api/recados/:id', (req, res) => {
    const recado = recados.find((r) => r.id === Number(req.params.id));
    if (!recado) return res.status(404).json({ erro: 'Recado não encontrado' })
;
    res.json(recado);
});

app.post('/api/recados', (req, res) => {
    const texto = lerTexto(req.body);
    if (!texto) return res.status(400).json({ erro: 'Escreva o texto do
recado' });
    const recado = { id: proximoId++, texto };
    recados.push(recado);
    res.status(201).json(recado);
});

app.put('/api/recados/:id', (req, res) => {
    const texto = lerTexto(req.body);
    if (!texto) return res.status(400).json({ erro: 'Escreva o texto do
recado' });
    const recado = recados.find((r) => r.id === Number(req.params.id));
    if (!recado) return res.status(404).json({ erro: 'Recado não encontrado' })
;
    recado.texto = texto;
    res.json(recado);
```

```
});

app.delete('/api/recados/:id', (req, res) => {
    const indice = recados.findIndex((r) => r.id === Number(req.params.id));
    if (indice === -1) return res.status(404).json({ erro: 'Recado não
encontrado' });
    const [removido] = recados.splice(indice, 1);
    res.json({ mensagem: 'Recado removido', recado: removido });
});

const PORT = process.env.PORT || 3001;
app.listen(PORT, (erro) => {
    if (erro) {
        console.error(`Não foi possível usar a porta ${PORT}: ${erro.message}`)
;
        process.exit(1);
    }
    console.log(`Mural no ar: http://localhost:${PORT}`);
});
```

Pontos do `app.js` que importam para a tela:

- `express.json()` converte o corpo JSON que o `fetch` envia em `req.body`. Sem essa linha, `req.body` chega vazio.
- `express.static(...)` serve a pasta `public`. Por isso, o endereço `http://localhost:3001` abre o `index.html`, mesmo sem uma rota escrita para ele.
- Os status seguem o contrato que as músicas também seguem: `200` na leitura, edição e exclusão; `201` no cadastro; `400` quando o texto falta; `404` quando o recado não existe. Todo erro vem como `{ erro: '...' }`. A tela mostra esse `erro`.
- `lerTexto` devolve `null` em três casos: o texto não veio, o texto não é `string`, ou o texto fica vazio depois do `trim`. A API valida de novo o que a tela já validou. A validação da tela é uma comodidade para quem digita. A validação da API é a garantia.
- O fim do arquivo usa `app.listen(PORT, (erro) => {...})`. No Express 5, o segundo argumento recebe o erro de inicialização. O caso mais comum é `EADDRINUSE`: a porta 3001 já está ocupada. Em geral, quem ocupa a porta é outro `npm start` que ficou rodando em outro terminal. Quando há erro de inicialização, o programa imprime a mensagem e encerra com `process.exit(1)`. Para resolver, escolham uma das opções:
    - Fechem o outro processo.
    - Rodem com outra porta. No Linux e no macOS: `PORT=3002 npm start`. No PowerShell: `$env:PORT=3002; npm start`.

Arquivo `public/index.html`

```html
<!DOCTYPE html>
<html lang="pt-BR">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Mural de recados</title>
    <link rel="stylesheet" href="estilo.css">
</head>
<body>
    <main>
        <h1>Mural de recados</h1>

        <form id="formulario-recado">
            <h2 id="titulo-formulario">Novo recado</h2>
            <label for="texto">Texto do recado</label>
            <input id="texto" name="texto" type="text" maxlength="200" autocomp
lete="off">
            <div class="acoes">
                <button id="salvar" type="submit">Cadastrar</button>
                <button id="cancelar-edicao" type="button" hidden>Cancelar
edição</button>
                <button id="excluir" type="button" disabled>Excluir
selecionado</button>
            </div>
            <p id="mensagem" role="status"></p>
        </form>

        <section>
            <h2>Recados</h2>
            <ul id="lista-recados"></ul>
        </section>
    </main>
    <script src="mural.js"></script>
</body>
</html>
```

Arquivo `public/estilo.css`

```css
/* estilo.css — visual simples do mural; funciona em tela estreita. */
* { box-sizing: border-box; }
body { margin: 0; font-family: 'Segoe UI', Arial, sans-serif; background: #f4f6
f8; color: #1f2933; }
main { max-width: 640px; margin: 0 auto; padding: 16px; }
h1 { color: #1d7a2d; }
form, section { background: #fff; border: 1px solid #d9e2e8; border-radius:
8px; padding: 16px; margin-bottom: 16px; }
label { display: block; font-weight: 600; margin-bottom: 4px; }
input { width: 100%; padding: 10px; font-size: 1rem; border: 1px solid #9aa8b2;
 border-radius: 6px; }
.acoes { display: flex; flex-wrap: wrap; gap: 8px; margin-top: 12px; }
button { padding: 10px 14px; font-size: 1rem; border-radius: 6px; border: 1px
solid #1d7a2d; background: #2f9e41; color: #fff; cursor: pointer; }
button[type="button"] { background: #fff; color: #1d7a2d; }
button:disabled { opacity: 0.5; cursor: not-allowed; }
button:focus-visible, input:focus-visible { outline: 3px solid #1d7a2d; outline
-offset: 2px; }
#mensagem { min-height: 1.4em; margin: 12px 0 0; }
#mensagem.sucesso { color: #1d7a2d; }
#mensagem.erro { color: #b00020; font-weight: 600; }
ul { list-style: none; padding: 0; margin: 0; }
li button { width: 100%; text-align: left; margin-bottom: 6px; background:
#fff; color: #1f2933; border-color: #d9e2e8; }
li.selecionado button { border: 2px solid #1d7a2d; background: #e6f4ed; }
```

O CSS não participa da lógica. Duas partes do CSS importam para os padrões:

- `#mensagem.sucesso` e `#mensagem.erro` pintam a mensagem de verde ou de vermelho, conforme a classe que o JavaScript coloca.
- `li.selecionado button` destaca o recado em edição.

Arquivo `public/mural.js`

```
// public/mural.js — tela do mural: lista, cadastra, edita e exclui recados
pela API.
const formulario = document.getElementById('formulario-recado');
const campoTexto = document.getElementById('texto');
const tituloFormulario = document.getElementById('titulo-formulario');
const botaoSalvar = document.getElementById('salvar');
const botaoCancelar = document.getElementById('cancelar-edicao');
const botaoExcluir = document.getElementById('excluir');
const mensagem = document.getElementById('mensagem');
const lista = document.getElementById('lista-recados');

let idEmEdicao = null;   // null = cadastrando; um número = editando esse
recado

// Mostra uma mensagem; erro = true pinta de vermelho
function mostrarMensagem(texto, erro) {
    mensagem.textContent = texto;
    mensagem.className = erro ? 'erro' : 'sucesso';
}

// Faz a requisição e devolve o JSON. Lança Error em dois casos:
// 1) o servidor respondeu com erro (400, 404, 500...)
// 2) não houve resposta (servidor parado, sem rede)
async function chamarApi(caminho, opcoes) {
    let resposta;
    try {
        resposta = await fetch(caminho, opcoes);
    } catch {
        throw new Error('Sem conexão com o servidor. Confiram se ele está
rodando.');
    }
    const dados = await resposta.json().catch(() => null);
    if (!resposta.ok) {
        throw new Error(dados?.erro || 'O servidor respondeu com erro ' +
resposta.status);
    }
    return dados;
}

// Desabilita os botões enquanto uma requisição está em andamento
function definirOcupado(ocupado) {
    botaoSalvar.disabled = ocupado;
    botaoCancelar.disabled = ocupado;
    botaoExcluir.disabled = ocupado || idEmEdicao === null;
}

// Busca os recados e redesenha a lista
```

```
async function carregarRecados() {
    const recados = await chamarApi('/api/recados');
    lista.innerHTML = '';
    for (const recado of recados) {
        const item = document.createElement('li');
        const botao = document.createElement('button');
        botao.type = 'button';
        botao.textContent = recado.texto;   // textContent: o texto aparece
como texto, nunca como HTML
        botao.addEventListener('click', () => selecionarRecado(recado));
        if (recado.id === idEmEdicao) item.className = 'selecionado';
        item.appendChild(botao);
        lista.appendChild(item);
    }
}

// Clicar num recado: o formulário passa para o modo edição
function selecionarRecado(recado) {
    idEmEdicao = recado.id;
    campoTexto.value = recado.texto;
    tituloFormulario.textContent = 'Editando recado ' + recado.id;
    botaoSalvar.textContent = 'Salvar alteração';
    botaoCancelar.hidden = false;
    botaoExcluir.disabled = false;
    mostrarMensagem('', false);
    carregarRecados().catch((erro) => mostrarMensagem(erro.message, true));   /
/ redesenha o destaque
}

// Volta o formulário para o modo cadastro
function voltarParaCadastro() {
    idEmEdicao = null;
    formulario.reset();
    tituloFormulario.textContent = 'Novo recado';
    botaoSalvar.textContent = 'Cadastrar';
    botaoCancelar.hidden = true;
    botaoExcluir.disabled = true;
}

// Submit do formulário: POST (cadastro) ou PUT (edição)
async function salvarRecado(evento) {
    evento.preventDefault();   // impede o navegador de recarregar a página
    const texto = campoTexto.value.trim();
    if (!texto) {
        mostrarMensagem('Escreva o texto do recado.', true);
        campoTexto.focus();
        return;
```

```
    }

    const editando = idEmEdicao !== null;
    const caminho = editando ? '/api/recados/' + idEmEdicao : '/api/recados';
    definirOcupado(true);
    try {
        const recado = await chamarApi(caminho, {
            method: editando ? 'PUT' : 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ texto })
        });
        voltarParaCadastro();
        await carregarRecados();
        mostrarMensagem((editando ? 'Recado alterado' : 'Recado cadastrado') +
' (id ' + recado.id + ').', false);
    } catch (erro) {
        mostrarMensagem(erro.message, true);   // o texto digitado continua no
campo
    } finally {
        definirOcupado(false);
    }
}

// Botão Excluir: confirma antes de enviar o DELETE
async function excluirRecado() {
    if (idEmEdicao === null) return;
    const confirmou = window.confirm('Excluir o recado ' + idEmEdicao + '?');
    if (!confirmou) {
        mostrarMensagem('Exclusão cancelada.', false);
        return;
    }
    definirOcupado(true);
    try {
        const resultado = await chamarApi('/api/recados/' + idEmEdicao, { metho
d: 'DELETE' });
        voltarParaCadastro();
        await carregarRecados();
        mostrarMensagem('Recado ' + resultado.recado.id + ' excluído.', false);
    } catch (erro) {
        mostrarMensagem(erro.message, true);
    } finally {
        definirOcupado(false);
    }
}

formulario.addEventListener('submit', salvarRecado);
botaoCancelar.addEventListener('click', () => {
```

```
    voltarParaCadastro();
    mostrarMensagem('Edição cancelada.', false);
});
botaoExcluir.addEventListener('click', excluirRecado);

carregarRecados().catch((erro) => mostrarMensagem(erro.message, true));
```

↑ Voltar ao sumário

Padrão 1 — HTML e JavaScript se encontram pelos IDs

O JavaScript não localiza os elementos do HTML pela posição nem pela aparência. Ele procura cada elemento por um **ID**, com `document.getElementById`, e guarda o resultado numa constante. O ID escrito no HTML e o texto procurado no JavaScript precisam ser idênticos. Se forem diferentes, a busca devolve `null`. Depois, a primeira linha que usar a constante falha com o erro `Cannot read properties of null`.

As primeiras linhas de `mural.js` fazem essa busca para cada elemento:

ID no `index.html`	Constante em `mural.js`	Papel
`formulario-recado`	`formulario`	O `<form>`; recebe o evento `submit` e o `reset()`.
`texto`	`campoTexto`	O campo de texto; `value` lê e escreve o que está digitado.
`titulo-formulario`	`tituloFormulario`	Muda entre "Novo recado" e "Editando recado N".
`salvar`	`botaoSalvar`	Botão `submit`; o texto alterna entre "Cadastrar" e "Salvar alteração".
`cancelar-edicao`	`botaoCancelar`	Aparece só no modo edição (atributo `hidden`).
`excluir`	`botaoExcluir`	Começa desabilitado; só fica habilitado com um recado selecionado.
`mensagem`	`mensagem`	Onde as mensagens de sucesso e de erro aparecem; tem `role="status"` para leitores de tela.
`lista-recados`	`lista`	O `<ul>` que a função de listar preenche.

A ordem do `<script>` importa. No `index.html`, a tag `<script src="mural.js">` fica no **fim do `body`**, depois de todos os elementos. O navegador lê o HTML de cima para baixo. Se o script estivesse no `head`, ele rodaria antes de o formulário existir. Nesse caso, todos os `getElementById` devolveriam `null`.

Como conferir.

1. Abram o console do navegador: apertem F12 e escolham a aba Console.
2. Digitem `document.getElementById('texto')` e apertem Enter.
3. Leiam o resultado. Se aparecer o elemento, o ID existe. Se aparecer `null`, o ID do HTML e o ID do JavaScript estão diferentes. Procurem diferença de maiúscula, de hífen ou de letra trocada.

↑ Voltar ao sumário

Padrão 2 — Listar e desenhar

A função `carregarRecados` faz três coisas, sempre nesta ordem: **buscar** os dados na API, **limpar** a lista antiga e **desenhar** um item para cada registro.

```
buscar:   const recados = await chamarApi('/api/recados');
limpar:   lista.innerHTML = '';
desenhar: para cada recado, criar <li> + <button>, preencher, anexar à lista
```

Pontos importantes:

- A função é `async` porque `fetch` é assíncrono. O `await` faz a função esperar a resposta antes de seguir.
- `lista.innerHTML = ''` apaga os itens antigos. Sem essa linha, cada chamada acrescentaria a lista inteira de novo, depois dos itens antigos.
- Cada item é criado com `document.createElement` e preenchido com **`textContent`**, não com `innerHTML`.

Por que `textContent` e não `innerHTML` para dados. O `innerHTML` interpreta o texto como HTML. O `textContent` mostra o texto como texto. Para dados que vêm de uma pessoa (o recado, o nome de uma música), só o `textContent` é seguro.

Teste do negrito. Cadastrem o recado `<b>negrito</b>`. Com `textContent`, a lista mostra exatamente `<b>negrito</b>`, com os sinais de menor e maior. Se vocês trocassem a linha por `botao.innerHTML = recado.texto`, a lista mostraria a palavra "negrito" em negrito. Isso aconteceria porque o navegador trataria o texto como marcação. Com um texto como `<img src=x onerror=...>`, a mesma troca faria o navegador executar código escrito por quem digitou o recado. Esse problema se chama injeção de HTML (XSS). O `textContent` evita o problema, e vocês não precisam escrever código a mais.

Observação sobre `selecionarRecado`. Cada clique num recado chama `carregarRecados()` de novo. O único objetivo é redesenhar o destaque do item selecionado. Isso funciona, mas gasta uma requisição por clique. Uma alternativa mais simples seria trocar a classe `selecionado` direto nos elementos da lista, sem chamar a API. O mural mantém o recarregamento por um motivo: assim, um único trecho de código mostra como a lista é sempre desenhada a partir do estado (`idEmEdicao`). Nas músicas, vocês podem escolher outra solução. A condição é que a lista mostrada e o item destacado correspondam sempre ao estado atual: os dados e o item selecionado.

↑ Voltar ao sumário

Padrão 3 — Cadastrar

O cadastro é o caso básico. Quando `idEmEdicao` é `null`, `salvarRecado` envia um `POST`. A sequência, linha a linha:

1. **`evento.preventDefault()`** é a primeira linha. Se o código não impedir, o `submit` de um formulário recarrega a página. Sem esta linha, acontecem três coisas: a página recarrega antes de a resposta da API chegar, a mensagem some e a lista é redesenhada desde o início.
2. **`campoTexto.value.trim()`** lê o texto e tira os espaços do início e do fim. Um texto só com espaços fica vazio.

3. **Validação antes de enviar.** Se o texto estiver vazio, a função mostra a mensagem de erro, coloca o foco no campo com `focus()` e termina com `return`. A função não faz nenhuma requisição. Isso atende ao R5 no lado da tela.
4. **O `fetch` do POST** recebe três opções:

Opção	Valor	Por quê
`method`	`'POST'`	Cadastrar.
`headers`	`{ 'Content-Type': 'application/json' }`	Avisa a API de que o corpo é JSON. Sem este cabeçalho, o `express.json()` ignora o corpo e a API responde `400`.
`body`	`JSON.stringify({ texto })`	O `fetch` só envia texto. `JSON.stringify` transforma o objeto em texto JSON.

5. **Status `201`.** A API responde `201 Created` com o recado criado. O código sabe se deu certo por `response.ok` (vejam o Padrão 4). O `response.ok` é `true` para qualquer status de `200` a `299`.
6. **Ler o `id` da resposta.** O recado devolvido traz o `id` que o servidor atribuiu. O mural usa esse `id` na mensagem: "Recado cadastrado (id 2)." Esse número prova que a API criou o registro. A tela não desenhou o item sozinha. Nas músicas, o R2 pede que vocês anotem esse `id`.
7. **Voltar ao cadastro e recarregar.** `voltarParaCadastro()` limpa o formulário e `await carregarRecados()` redesenha a lista com o novo registro.

Dois detalhes de ordem.

1. A função só escreve a mensagem de sucesso **depois** de `chamarApi` retornar sem lançar erro.
2. A função escreve a mensagem **depois** de recarregar a lista.

Consequência do segundo detalhe: a gravação pode funcionar e o recarregamento da lista pode falhar. Isso acontece se a conexão cair entre as duas chamadas. Nesse caso, a tela mostra a mensagem de falha de conexão, mas o recado foi gravado. A mensagem informa corretamente o que falhou: a atualização da lista. Mesmo assim, ela pode confundir.

Se uma mensagem de erro aparecer logo depois de um salvamento, façam assim:

1. Abram a aba Rede.
2. Leiam a ordem das requisições.
3. Se o `POST` aparece com `201`, o registro existe. Só a lista não atualizou.

> **Pergunte ao agente.** *"Explique o que acontece se eu esquecer o evento.preventDefault() no submit. Antes de responder, peça que eu preveja. Use como fonte o guia_interface_crud.md (Padrão 3), na página do trabalho: https://elderbernardiifsul.github.io/programacao-iii/integrador-2026/guia_interface_crud.md."*

↑ Voltar ao sumário

Padrão 4 — Erro HTTP × falha de conexão

O `fetch` tem uma regra que surpreende quem vem de outras linguagens: **ele só lança exceção quando não recebe resposta do servidor.** Se o servidor respondeu, mesmo com `400`, `404` ou `500`, o `fetch` termina normalmente. Ele entrega uma resposta em que `response.ok` é `false`. O código de vocês é que precisa conferir o status.

A função `chamarApi` trata os dois casos do mesmo jeito: ela lança um `Error` com uma mensagem que a tela pode mostrar. Linha a linha:

```
async function chamarApi(caminho, opcoes) {
    let resposta;                                   // declarada fora do try para
valer depois dele
    try {
        resposta = await fetch(caminho, opcoes);    // falha aqui = sem resposta
nenhuma
    } catch {
        throw new Error('Sem conexão com o servidor. Confiram se ele está
rodando.');
    }
    const dados = await resposta.json().catch(() ⇒ null);   // corpo não é
JSON? vira null
    if (!resposta.ok) {                             // o servidor respondeu, mas
com erro
        throw new Error(dados?.erro || 'O servidor respondeu com erro ' +
resposta.status);
    }
    return dados;                                   // só chega aqui se deu certo
}
```

- O `try/catch` em volta do `fetch` captura apenas a **falha de conexão**.
- `resposta.json().catch(() ⇒ null)` evita um segundo erro quando o corpo da resposta não é JSON (por exemplo, uma página de erro em HTML). Nesse caso, `dados` fica `null`.
- `dados?.erro` usa o campo `erro` que a API manda. Se ele não existir, a mensagem usa o texto genérico com o status.
- A função que chama `chamarApi` usa um único `try/catch` e trata os dois casos do mesmo jeito: `mostrarMensagem(erro.message, true)`.

Situação, `fetch` e tela

Situação	O que o `fetch` faz	O que a tela mostra
`400` — um texto vazio chegou à API apesar da validação da tela	Termina normal; `response.ok` é `false`	A mensagem da API: "Escreva o texto do recado", em vermelho
`404` — recado já excluído em outra aba	Termina normal; `response.ok` é `false`	"Recado não encontrado", em vermelho
`500` — erro interno no servidor	Termina normal; `response.ok` é `false`	A mensagem de `erro`, se o servidor mandar; senão "O servidor respondeu com erro 500"

Situação	O que o `fetch` faz	O que a tela mostra
Servidor parado ou sem rede	**Lança exceção** (o `catch` do `try` interno)	"Sem conexão com o servidor. Confiram se ele está rodando.", em vermelho
`200` ou `201`	Termina normal; `response.ok` é `true`	Mensagem de sucesso, em verde

A regra: a tela só anuncia sucesso depois da resposta `ok`. No código, a linha `mostrarMensagem('Recado cadastrado...', false)` fica **depois** do `await chamarApi(...)`, nunca antes. Se a linha ficasse antes, uma queda de conexão faria a tela dizer "cadastrado" para um registro que não existe. Esse é o erro que o R5 pede para evitar.

> **Pergunte ao agente.** *"Qual a diferença entre a resposta ter response.ok falso e o fetch lançar uma exceção? Dê um exemplo de cada situação no meu projeto. Use como fonte o guia_interface_crud.md (Padrão 4), na página do trabalho: https://elderbernardiifsul.github.io/programacao-iii/integrador-2026/guia_interface_crud.md."*

↑ Voltar ao sumário

Padrão 5 — Selecionar e editar

A edição não usa uma tela nova. Ela usa o **mesmo formulário**, em outro modo. Uma única variável de estado, `idEmEdicao`, indica o modo atual do formulário:

`idEmEdicao`	Modo	Título	Botão	Cancelar	Excluir
`null`	Cadastro	Novo recado	Cadastrar	escondido	desabilitado
um número	Edição	Editando recado N	Salvar alteração	visível	habilitado

As três funções que usam esse estado:

- **`selecionarRecado(recado)`** coloca o formulário no modo edição. Ela guarda o `id`, copia o texto para o campo, troca o título e o texto do botão, mostra "Cancelar edição" e habilita "Excluir selecionado". No fim, ela recarrega a lista para redesenhar o destaque (vejam a observação do Padrão 2).
- **`voltarParaCadastro()`** desfaz essas mudanças: `idEmEdicao = null`, `formulario.reset()` e os textos originais. O mural chama essa função depois de salvar, depois de excluir e ao cancelar.
- **`salvarRecado`** escolhe entre criar e alterar:

```
const editando = idEmEdicao !== null;
const caminho  = editando ? '/api/recados/' + idEmEdicao : '/api/recados';
method: editando ? 'PUT' : 'POST'
```

  O mesmo `submit` serve aos dois modos. O que muda é o método (`PUT` ou `POST`) e o endereço (com ou sem `/id`). Por isso, a variável `idEmEdicao` precisa ter o valor certo na hora de salvar. Se ela estiver `null` no modo edição, o `submit` cria um registro novo em vez de alterar o existente.

O botão **Cancelar edição** chama `voltarParaCadastro()` e informa "Edição cancelada." O botão não envia nada à API.

Com a API parada, o texto digitado continua no campo. No `catch` de `salvarRecado`, a função não chama `voltarParaCadastro()`. O texto digitado continua no campo e o modo edição continua ativo. Assim, a pessoa pode tentar de novo.

> **Pergunte ao agente.** *"Estou no modo edição e, ao salvar, é criada uma música nova em vez de alterar a existente. Que perguntas devo me fazer para achar o erro? Use como fonte o guia_interface_crud.md (Padrão 5 e "Aplicando às músicas"), na página do trabalho: https://elderbernardiifsul.github.io/programacao-iii/integrador-2026/guia_interface_crud.md."*

↑ Voltar ao sumário

Padrão 6 — Excluir com confirmação

`excluirRecado` segue uma ordem fixa:

1. **Exigir seleção.** Se `idEmEdicao` for `null`, a função termina na primeira linha. Sem seleção, o botão Excluir já fica desabilitado. Esta verificação é uma segunda proteção, dentro da função.
2. **Confirmar.** `window.confirm('Excluir o recado N?')` abre uma caixa com OK e Cancelar e devolve `true` ou `false`. Se for `false`, a função mostra "Exclusão cancelada." e termina **sem enviar nada**. Esse é o comportamento que o R4 pede: cancelar mantém o registro.
3. **Enviar o `DELETE`** para `/api/recados/` + o `id`. A requisição não tem corpo.
4. **Atualizar.** Depois de uma resposta `ok`, acontecem três coisas, nesta ordem:
    - `voltarParaCadastro()` limpa a seleção.
    - `await carregarRecados()` redesenha a lista sem o registro.
    - A mensagem usa o que a API devolveu: `resultado.recado.id`.

    A API do mural responde `{ mensagem, recado }`. Para as músicas, o contrato é `{ mensagem, musica }`.

Observação sobre o foco. A correção descrita aqui é opcional. Os requisitos não exigem.

- Durante a exclusão, `definirOcupado(true)` desabilita o botão Excluir.
- No `finally`, o botão continua desabilitado, porque `voltarParaCadastro()` já limpou a seleção.
- O botão que tinha o foco deixou de aceitar foco. Por isso, quem navega pelo teclado perde a posição na página.

O mural não corrige esse problema. Se vocês quiserem corrigir, mandem o foco para o campo de texto depois de excluir. Se houver tempo, vocês podem fazer essa correção nas músicas.

> **Pergunte ao agente.** *"Revise a minha função de exclusão (colada abaixo). Aponte a linha do problema e explique, sem reescrever a função. Use como fonte o guia_interface_crud.md (Padrão 6), na página do trabalho: https://elderbernardiifsul.github.io/programacao-iii/integrador-2026/guia_interface_crud.md."*

↑ Voltar ao sumário

Padrão 7 — Evitar envio duplicado

Se a pessoa clicar duas vezes em "Cadastrar" antes de a resposta chegar, a página envia duas requisições e a API cria dois registros. Para evitar isso, o mural desabilita os botões enquanto a requisição está em andamento.

`definirOcupado(ocupado)` faz isso em um lugar só:

```
botaoSalvar.disabled   = ocupado;
botaoCancelar.disabled = ocupado;
botaoExcluir.disabled  = ocupado || idEmEdicao === null;
```

A última linha tem uma condição a mais. Quando a requisição termina (`ocupado` igual a `false`), o botão Excluir só volta a ficar habilitado se ainda houver um recado selecionado.

O padrão fica completo com o `finally` :

```
definirOcupado(true);
try {
    ...await chamarApi(...)...
} catch (erro) {
    mostrarMensagem(erro.message, true);
} finally {
    definirOcupado(false);     // roda com sucesso OU com erro
}
```

O `finally` roda sempre, inclusive quando o `catch` é executado. Se o `definirOcupado(false)` estivesse só no fim do `try` , um erro de rede deixaria os botões desabilitados até a pessoa recarregar a página. Esse é o sintoma "o botão Excluir continua desabilitado" da seção "Se deu errado".

↑ Voltar ao sumário

Conferir na aba Rede (DevTools)

A tela mostra o que o código JavaScript mandou mostrar. A aba **Rede** (em inglês, *Network*) mostra as requisições e as respostas que realmente passaram entre o navegador e a API. Usem a aba Rede para obter a evidência do Percurso para testar e demonstrar.

Abrir.

1. Abram a página do mural no navegador.
2. Apertem F12.
3. Escolham a aba Rede.
4. Marquem o filtro **Fetch/XHR**. Com esse filtro, a aba mostra só as chamadas à API.
5. Recarreguem a página (F5). Assim, a lista de requisições recomeça a partir desse recarregamento.

O que ler em cada requisição. Cliquem numa linha da lista de requisições. A tabela abaixo mostra onde olhar:

Onde olhar	O que mostra
Coluna **Método** (ou aba Cabeçalhos)	`GET`, `POST`, `PUT` ou `DELETE`
Coluna **Nome** / URL	O endereço chamado, como `/api/recados/2`
Coluna **Status**	`200`, `201`, `400`, `404`...
Aba **Carga** (*Payload*)	O que a página enviou no corpo (o JSON de `JSON.stringify`)
Aba **Resposta** (*Response*)	O que a API devolveu, como texto JSON

Onde ver o `id` criado (evidência do R2).

1. Cadastrem um recado.
2. Na aba Rede, procurem a requisição `POST` para `/api/recados` e cliquem nela.
3. Abram a aba Resposta. O JSON traz `id` e `texto`. Esse `id`, junto com o status `201`, mostra que a API criou o registro.
4. Anotem o `id`. O Percurso pede que vocês consultem o registro de novo depois.

Falha controlada (R5).

1. Com a página aberta e funcionando, voltem ao terminal onde o `npm start` roda e apertem `Ctrl+C`. O servidor para. A página continua aberta.
2. Na página, digitem um recado e cliquem em "Cadastrar".
3. Leiam a mensagem. Ela deve ser a de falha de conexão ("Sem conexão com o servidor...") e **não** pode dizer que o recado foi cadastrado. Na aba Rede, a requisição aparece em vermelho, sem status.
4. Reiniciem com `npm start`, recarreguem a página e confiram a lista. Como os recados ficam na memória, só o recado inicial volta.

Esse é o mesmo teste da "Falha controlada local" do enunciado do projeto. Nas músicas, a diferença é que a lista volta com os dados do banco.

↑ Voltar ao sumário

Aplicando às músicas (sem código)

Esta seção diz **o que muda** do mural para o catálogo e **o que conferir** em cada requisito. Ela não traz o código das músicas. Vocês escrevem esse código, usando os padrões acima.

O que muda do mural para as músicas

Ponto	Mural de recados	Músicas
Campos do formulário	Um (`texto`)	Dois (`nome` e `artista`), ambos obrigatórios
Corpo enviado no POST e no PUT	`{ texto }`	`{ nome, artista }`, os dois como **texto**
O que a API devolve para o artista	—	Um **objeto** `{ id, nome }`. Para preencher o campo no modo edição, usem `artista.nome`. Usar `artista` direto escreve `[object Object]` no campo

Ponto	Mural de recados	Músicas
Editar o artista	—	Muda o **vínculo desta música**: a música passa a apontar para outro artista (que a API encontra ou cria pelo nome). Os outros registros do artista antigo não são renomeados
Lista	Só o texto	Cada item traz também `totalPartes`
Música sem partes	—	É válida. Ao selecioná-la, a tela informa que não há partes e mantém **Tocar** indisponível
Resposta do DELETE	`{ mensagem, recado }`	`{ mensagem, musica }`
Validação na tela	Um campo vazio	Nome vazio **ou** artista vazio: cada um deve ser recusado; de preferência, a mensagem diz qual

Checklist por requisito

Os testes abaixo vêm do "Percurso para testar e demonstrar" do enunciado de "O repertório da turma". Em todos os testes, usem uma música de teste sem partes. Preservem a música que o player usa na demonstração: ela precisa continuar cadastrada e tocando.

Requisito	O que a tela precisa fazer	Como conferir
R2 — Cadastrar	Enviar nome e artista (`POST /api/musicas`); mostrar o resultado; recarregar a lista	Cadastrar uma música de teste. Na aba Rede, ver `201` e anotar o `id` da resposta. Consultar a música de novo. Ao selecioná-la, a tela informa que não há partes e Tocar fica indisponível
R3 — Editar	Preencher o formulário ao selecionar; enviar `PUT /api/musicas/:id` com o `id` em edição; manter o `id`	Mudar nome ou artista da música de teste. A lista e a resposta da API mostram a mudança; o `id` é o mesmo. Reiniciar o servidor e consultar de novo: o mesmo `id` com os dados novos
R4 — Excluir	Exigir seleção; pedir confirmação; só então enviar `DELETE /api/musicas/:id`	Iniciar a exclusão e **cancelar**: a consulta seguinte mostra a música ainda cadastrada. Iniciar de novo e **confirmar**: a consulta seguinte não a mostra
R5 — Validar e informar	Recusar campos vazios antes de enviar; mostrar erro em vermelho e sucesso em verde; distinguir erro da API de falha de conexão	Tentar cadastrar com nome vazio e com artista vazio: a tela informa o problema e não anuncia cadastro. Parar o servidor local, tentar atualizar a lista ou cadastrar: a mensagem de erro aparece, nunca a de sucesso. Reiniciar e consultar a lista

Quem usa a base de retomada

A base de retomada está disponível no Classroom da disciplina e em base_retomada.zip. Quem usa a base de retomada não escreve a página inteira. Todo o código de vocês fica em **`public/admin.js`**. Esse arquivo já está ligado ao HTML e traz comentários `TODO` para cadastro, edição e exclusão. O `catalogo.js` já oferece os itens da tabela abaixo:

Item pronto	O que faz
`consultarApi(caminho, opcoes)`	Faz o `fetch`, devolve o JSON em caso de sucesso e **lança** `Error` tanto para erro HTTP quanto para falha de rede. É o equivalente do `chamarApi` do mural
`carregarMusicas()`	Busca as músicas e redesenha a lista (o equivalente de `carregarRecados`)
`limparSelecao()`	Desfaz a seleção atual

Item pronto	O que faz
`mostrarMensagem(id, texto, erro)`	Escreve uma mensagem no elemento de ID `id`; `erro` pinta como erro. Atenção: recebe o **ID do elemento** como primeiro argumento, ao contrário do mural
`definirCatalogoOcupado(bool)`	Trava e destrava a lista enquanto uma operação está em andamento
`estadoCatalogo.musicaSelecionada`	O detalhe da música selecionada, ou `null`
evento `musica-selecionada`	Disparado ao selecionar ou limpar; o detalhe vem em `evento.detail`, ou `null`

Os IDs do formulário da base: `formulario-musica`, `nome`, `artista`, `salvar-musica`, `excluir-musica`, `nova-musica`, `titulo-formulario` e `estado-formulario`.

Correspondência entre os padrões do mural e a base:

- O `idEmEdicao` do mural corresponde ao ID da música selecionada.
- O `voltarParaCadastro` corresponde a preparar um novo cadastro: limpar a seleção e os campos.
- O `definirOcupado` corresponde a duas ações juntas: a trava dos botões do formulário e o `definirCatalogoOcupado`.

Os controles de escrita vêm desabilitados no HTML. Habilitar o botão não basta. Vocês precisam integrar a operação e tratar a resposta.

> **Cuidado ao aproveitar o código do mural.** O `admin.js` divide o escopo global com outro arquivo: o `catalogo.js` (na base) ou o `player.js` (no projeto do Guia passo a passo do Supabase). Por isso, o que um arquivo declara no nível superior vale para todos os scripts da página.
>
> Dois problemas que isso causa:
>
> - Se vocês copiarem a `mostrarMensagem(texto, erro)` do mural para o `admin.js` da base, ela substitui a `mostrarMensagem(id, texto, erro)` do `catalogo.js`. As mensagens do catálogo param de funcionar.
> - Se vocês repetirem um `const` ou `let` de nível superior que já existe no outro arquivo, o console mostra o erro `Identifier '…' has already been declared`. O `admin.js` inteiro deixa de rodar.
>
> Usem as funções que a base já oferece. Escolham nomes novos para o que vocês criarem.

Quem seguiu o Guia passo a passo do Supabase

Quem seguiu o "Guia passo a passo do Supabase" tem a página em `public/index.html` e o código em `public/player.js`. Os IDs do player são `listaMusicas`, `nomeMusica`, `artista`, `tag`, `letra`, `contador`, `btnTocar` e `mensagem`. Para fazer as operações de escrita, sigam estes passos:

1. Acrescentem ao `public/index.html` um `<form>` com os campos `nome` e `artista`, um elemento de mensagem e os botões (cadastrar, cancelar edição e excluir). Cuidado com os IDs: `artista` e `mensagem` **já existem** no player. Dois elementos com o mesmo ID quebram o `getElementById`. Escolham IDs novos para os elementos do formulário.
2. Criem o arquivo `public/admin.js`.

3. Incluam o `public/admin.js` no `index.html` **depois** do `player.js`, os dois no fim do `body`. O cuidado com o escopo global, explicado na seção anterior, vale aqui também: não repitam no `admin.js` um nome declarado no nível superior do `player.js`.
4. Para recarregar a lista depois de cada operação, chamem `carregarPlaylist()`. Essa é a função do `player.js` que desenha a lista.
5. Apliquem os Padrões 3 a 7 com o mesmo formato de chamada do mural. Troquem `/api/recados` por `/api/musicas`.

↑ Voltar ao sumário

Se deu errado

Sintoma	Causa provável	O que fazer
A página recarrega ao salvar e a mensagem some	Faltou `evento.preventDefault()` no `submit`, ou o listener foi ligado ao botão em vez de ao formulário	Confiram a primeira linha da função do `submit` e `formulario.addEventListener('submit', ...)` (Padrão 3)
A tela diz "cadastrado" com o servidor parado	A mensagem de sucesso é escrita antes da resposta, ou o `fetch` é chamado sem tratar a exceção	Confiram se o código só escreve a mensagem de sucesso depois do `await` que retornou sem lançar erro (Padrão 4)
O campo artista mostra `[object Object]`	O campo recebeu o objeto `artista` em vez de `artista.nome`	Preencham o campo com `musica.artista.nome` (vejam a tabela "O que muda")
A lista não atualiza depois de salvar	Faltou chamar (ou esperar) o recarregamento da lista depois do salvamento	Chamem a função de listar com `await` depois da operação. Confiram na aba Rede se o `GET` saiu depois do `POST`
O botão Excluir continua desabilitado	O `definirOcupado(false)` não roda em caso de erro, ou a seleção foi limpa	Coloquem o `definirOcupado(false)` dentro do `finally` (Padrão 7) e confiram o estado da seleção
`Cannot read properties of null` na primeira linha	ID do JavaScript diferente do ID do HTML, ou `<script>` antes dos elementos	Confiram o ID no console com `getElementById` e a posição do `<script>` (Padrão 1)
A API responde `400` mesmo com o campo preenchido	Faltou o cabeçalho `Content-Type: application/json` ou o `JSON.stringify`	Abram a aba Rede e, na requisição, a aba Carga. O corpo deve ser um JSON com os campos
No console: `Identifier '…' has already been declared`, e nenhum botão do formulário responde	Um `const`, `let` ou função de nível superior do `admin.js` repete um nome do `catalogo.js` ou do `player.js`	Renomeiem o que vocês criaram no `admin.js` (seção "Quem usa a base de retomada")
`EADDRINUSE` ao iniciar o servidor	A porta já está ocupada por outro `npm start`	Fechem o outro processo ou usem outra porta (seção "Os cinco arquivos")

↑ Voltar ao sumário